Listas Circulares

Professor: André Felipe Ribeiro Cordeiro
Disciplina: Estruturas de Dados

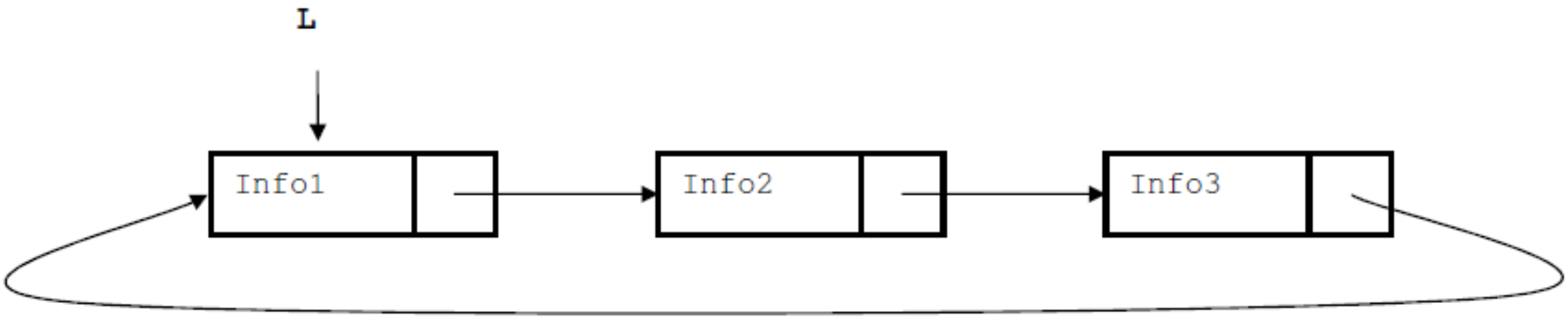
Departamento de Informática
Centro de Tecnologia
Universidade Estadual de Maringá

Introdução

- Diferentes aplicações podem exigir a **representação cíclica de dados**. Nesses casos, pode-se utilizar **listas circulares**.

- Em uma lista circular, o **último elemento** da lista está relacionado com o **primeiro elemento**, formando um **ciclo**.

Representação



Percurso

- Para o **percurso completo** em uma lista circular, inicia-se a visita a **partir de um elemento**.

- O percurso acontece até que esse mesmo elemento seja **alcançado**.

- **Ponteiros** podem ser utilizados para referenciar tanto o **primeiro** quanto o **último** elemento da lista.

Operações
Listas Circulares

- As seguintes operações podem ser utilizadas em diferentes aplicações:
  - **inicializar** a lista
  - **verificar** se a lista está **vazia**
  - **inserir item** na lista
  - **retirar item** da lista
  - **imprimir os itens** da lista

Implementação

```
#include <stdlib.h>
#include <stdio.h>
```

```
typedef struct TipoItem *TipoApontador;

typedef struct TipoItem {
  int conteudo;
  TipoApontador proximo;
} TipoItem;
```

Implementação

```
typedef struct {
  TipoApontador primeiro, ultimo;
} TipoLista;
```

```
void Inicializa(TipoLista *lista){
  lista->primeiro =
       (TipoApontador) malloc (sizeof(TipoItem));
  lista->primeiro->proximo = lista->primeiro;
  lista->ultimo = lista->primeiro;
}
```

Implementação

```
int Vazia(TipoLista lista){
  return (lista.primeiro == lista.ultimo);
}
```

```
void Insere(int elemento, TipoLista *lista){
  lista->ultimo->proximo =
       (TipoApontador) malloc (sizeof(TipoItem));
  lista->ultimo = lista->ultimo->proximo;
  lista->ultimo->conteudo = elemento;
  lista->ultimo->proximo = lista->primeiro;
}
```

Implementação

```
void Retira(TipoApontador ponteiro,
                                TipoLista *lista, int elemento){
  TipoApontador p_auxiliar;
  if (Vazia(*lista) || ponteiro == NULL) {
    printf(" Erro! Lista vazia ou posicao nao existe \n");
    return;
  }
  p_auxiliar = ponteiro->proximo;
  elemento = p_auxiliar->conteudo;
  ponteiro->proximo = p_auxiliar->proximo;

  if (ponteiro->proximo == lista->primeiro)
    lista->ultimo = ponteiro;
  free(p_auxiliar);
}
```

Implementação

```
void Imprime(TipoLista lista){
  TipoApontador p_auxiliar;
  p_auxiliar = lista.primeiro->proximo;

  while (p_auxiliar != lista.primeiro) {
    printf("%d\n", p_auxiliar->conteudo);
    p_auxiliar = p_auxiliar->proximo;
  }
}
```

Observações

- Outras operações poderiam ser consideradas, além das apresentadas.

- **Exemplos**
  - **Pesquisar** um item da lista
  - **Contar** o número de itens repetidos da lista

Referências

- Introdução a Estruturas de Dados, Waldemar Celes, Renato Cerqueira e José Rangel, 2016.

- Projeto de Algoritmos com implementações em Pascal e C, Nivio Ziviani, 2011.

- Aulas Professor Rodolfo Miranda Pereira, UEM.